This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.



# An Integrated Sensing-Computing Processor for Spectral-Based Chinese Herbal Medicine Classification With Algorithm-FPGA Co-Design

Hui Zhang, *Member, IEEE*, Hai Wang, Bingrui Zhao, *Student Member, IEEE*, Tongzhi Niu, Yunkang Cao, *Graduate Student Member, IEEE*, and Yaonan Wang, *Senior Member, IEEE*

***Abstract*—Hyperspectral imaging has demonstrated strong potential for the classification of Chinese herbal medicines (CHM). However, conventional push-broom systems rely on back-end PCs for spectral analysis, resulting in substantial data transfer overhead and limiting their applicability in real-time, energy-efficient edge scenarios. To address this, we propose an integrated sensing–computing processor. A complete hyperspectral imaging electronics system is developed, along with an dual-branch model, push-broom vision transformer (PBViT), which combines a frequency-domain encoding branch with an improved ViT architecture and contains only 0.075 MB of parameters. On the CHM dataset, PBViT achieves state-of-the-art performance with an OA of 0.978, AA of 0.942, and Kappa of 0.950, surpassing traditional machine learning, convolutional, and ViT-based methods. Leveraging quantization-aware training (QAT), the model is compressed to 8-bit precision with negligible accuracy loss. We further develop an FPGA-based accelerator for PBViT, achieving 502 GOPS throughput and 32.2 $\mu$s latency per $9 \times 9$ patch, with a power consumption of 14.42 W. This yields energy efficiency improvements of 6.24$\times$ and 63$\times$ over an RTX 4090 Laptop GPU and an Intel i9-13900HX CPU, respectively. The proposed processor delivers high efficiency for real-time, edge-side pharmaceutical identification and can also be applied to fields such as the food industry and agriculture.**

***Index Terms*—Algorithm-FPGA co-design, push-broom imaging, spectral camera processor, spectral image classification, ViT accelerator.**

Received 27 July 2025; revised 4 November 2025; accepted 24 November 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62027810, in part by the Major Research Plan of the National Natural Science Foundation of China under Grant 92148204, and in part by the Top Ten Technical Research Projects of Hunan Province under Grant 2024GK1010. *(Corresponding author: Hui Zhang.)*

The authors are with the School of Artificial Intelligence and Robotics and the National Engineering Laboratory for Robot Visual Perception and Control Technology, Hunan University, Changsha 410012, China (e-mail: zhanghui1983@hnu.edu.cn; whai@hnu.edu.cn; barryzhao@hnu.edu.cn; niutongzhi@hnu.edu.cn; caoyunkang@hnu.edu.cn; yaonan@hnu.edu.cn).

Digital Object Identifier 10.1109/TIE.2025.3639807

## I. Introduction

HYPERSPECTRAL imaging systems, characterized by their high spatial resolution combined with continuous spectral sampling, have proven highly effective for the noncontact and nondestructive classification of CHM [1], [2]. Conventional push-broom systems generally employ a decoupled sensing-computing architecture, where the captured spectral data is transferred to high-performance industrial PCs for subsequent processing. In these systems, FPGA is typically used as the deployment platform to handle image acquisition, data transmission, and sensor control, without integrating on-chip computation. However, this approach introduces substantial bandwidth and latency overheads and relies on power-hungry general-purpose processors, restricting its suitability for real-time, energy-constrained applications. Consequently, there is a clear need for an integrated sensing-computing processor that handles on-chip analysis, acquisition, transmission, control, and all other operations, eliminating off-board data transmission, thereby offering real-time performance, a lightweight design, and low power consumption [3], [4].

To overcome these limitations, various hardware platforms including CPUs, GPUs, and ASICs have been explored for spectral data processing, each presenting distinct tradeoffs in computational efficiency, energy consumption, and implementation costs [5]. Among these, FPGAs are particularly attractive due to their reconfigurable logic, fine-grained parallelism, and low power consumption, making them ideal for integrated sensing-computing systems [6], [7]. Existing FPGA-based HSI processors typically utilize traditional analytical techniques. For instance, Gonzalez et al. [8] developed a parallel FPGA implementation of the image space reconstruction algorithm for hyperspectral unmixing, achieving near real-time estimation of endmember abundances. Similarly, Macias et al. [9] presented a hardware-optimized approach for automatic target detection and classification using Gram-Schmidt orthogonalization. Palacios et al. [10] introduced a real-time FPGA implementation of the linearly constrained minimum variance algorithm, utilizing Lapack D factorization decomposition and incremental correlation updates to satisfy the rigorous resource and latency requirements of unmanned aerial vehicle (UAV)-based deployments. Additionally, Morcillo et al. [11] described an

1557-9948 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:17:17 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.



RTL-level FPGA architecture for real-time $k$-means clustering in HSI classification. Although these methods achieve real-time processing, their reliance on traditional analytical approaches restricts their effectiveness in capturing the hierarchical spatial-spectral features necessary for CHM classification.

More recently, deep learning techniques, especially ViT-based models, have demonstrated superior capabilities in capturing spatial-spectral dependencies, crucial for accurately classifying CHM [12], [13]. Hong et al. [14] proposed Spectralformer, a ViT-based backbone for HSI classification, which incorporates cross-layer skip connections to enhance data propagation and achieve superior performance. Sun et al. [15] introduced SSFTT, a model that combines spatial-spectral feature extraction with a Gaussian-weighted feature tokenizer. By integrating 2-D and 3-D convolutional modules for initial feature processing and leveraging a transformer encoder for high-level semantic learning, SSFTT demonstrates notable improvements in classification accuracy. Zhao et al. [16] introduced GSCViT, which uses a groupwise separable convolution module for local spatial-spectral feature extraction and a groupwise separable multihead self-attention module to capture both local and global features, achieving superior performance with fewer training samples. Zhang et al. [2] proposed S2F$^2$AN, a model for CHM segmentation that incorporates spatial-spectral frequency attention and cross-feature fusion mechanisms to enhance feature extraction, resulting in superior performance.

By combining the expressive power of deep learning with the low latency provided by FPGA parallelism, there is growing interest in deploying neural networks on FPGA. For example, Pitonak et al. [17] introduced CloudSatNet-1, a quantized CNN tailored for onboard cloud classification in nanosatellites, which enables real-time inference with minimal power and memory consumption. He et al. [18] devised a hybrid 2-D-3-D CNN architecture complemented by a bespoke data access and memory scheduling strategy, achieving high-throughput parallel acceleration on FPGA. Furthermore, Baba et al. [19] proposed a framework based on wavelet-compressed CNNs for spectral feature extraction and classification, which was efficiently deployed on FPGA through parallel optimization techniques. Despite these advancements, FPGA-based deep learning approaches predominantly focus on CNNs and RNNs, leaving ViT-based architectures, noted for their intricate attention mechanisms and complex computational patterns, largely unexplored in FPGA implementations. Therefore, there is an urgent need for algorithm-hardware co-design strategies to balance ViT model performance and FPGA implementation feasibility, particularly in edge-oriented CHM applications. Moreover, the vast majority of methods employ HLS as the deployment language, which limits the optimization of the accelerator's hardware resources and timing.

To tackle these challenges, we have developed an integrated sensing-computing processor on FPGA, paired with a ViT-based hyperspectral model named PBViT. By employing an Algorithm-FPGA co-design approach, we have optimized both the model architecture and the hardware accelerator to deliver high-throughput, low-latency, and energy-efficient spectral analysis directly on the chip. The principal contributions of this work are outlined below.

1) A push-broom hyperspectral electronic system comprising a core board, an interface board, and a sensor board, seamlessly integrated for rapid spectral data acquisition, reliable data transmission, and real-time classification that is synchronized with the push-broom scanning mechanism.
2) PBViT, an innovative, algorithm-FPGA co-design model for CHM classification, uniquely combines frequency-domain encoding with transformer architecture to enhance spectral discrimination while maintaining low resource consumption and computational latency. The model undergoes compression using QAT into an 8-bit integer format for efficient FPGA deployment.
3) An FPGA-based accelerator designed explicitly for PBViT, marking the first FPGA-based ViT acceleration architecture dedicated to HSI classification. The accelerator employs an outer-product matrix multiplication structure and unified nonlinear computational cores, achieving inference times of 32.2 $\mu$s for a $9 \times 9$ patch at 300 MHz. This yields a throughput of 502.3 GOPS, comparable to 76% of an NVIDIA RTX 4090 Laptop GPU and 7.1$\times$ faster than an Intel i9-13900HX CPU.

This article is organized as follows. Section II provides an introduction to the electronic system, Section III details the design, quantization, and implementation of PBViT, and Section IV presents the experimental results. The abbreviated descriptions of the terms are provided in Table I.

## II. Spectral Camera Electronic System

This section introduces the hardware design of the spectral camera electronic system and provides a description of the system's functionality.

### A. Hardware Design

The spectral camera electronic system consists of three components: the core board, the interface board, and the sensor board. The core board, shown in Fig. 1(a), serves as the central unit of the system. It features the ZU19EG device, which integrates both an FPGA and a PS with four ARM cores. Both the FPGA and the PS are equipped with 8 GB of 1600 MHz MT40A1G16RC DDR4 memory. The core board integrates the LTM4700Y main power supply IC, which powers all voltage domains. Additionally, it features an PHY chip supporting gigabit Ethernet and 10G SFP+ ports for high-speed data transfer. The ERM8 connector is used to interface with other boards. For storage, the core board incorporates QSPI and eMMC chips for program booting and configuration.

The interface board provides essential physical-layer connectivity, including an RJ45 Ethernet port, a six-pin JTAG interface for SoC debugging, an STLINK interface for STM32F103 programming and debugging, a USB UART interface, a 12 V dc power input, and an SD card slot, as shown in Fig. 1(b) and (d). Additionally, the STM32F103 MCU is integrated into the interface board to manage the system's power-on sequence.

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:17:17 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.



TABLE I
TABLE OF ABBREVIATIONS

| Abbreviation | Full Form |
|---|---|
| CHM | Chinese Herbal Medicines |
| HSI | Hyperspectral Image |
| PC | Personal Computer |
| FPGA | Field-Programmable Gate Array |
| CPU | Central Processing Unit |
| GPU | Graphics Processing Unit |
| ASIC | Application-Specific Integrated Circuit |
| CIS | CMOS Image Sensor |
| RTL | Register Transfer Level |
| HLS | High-Level Synthesis |
| FSM | Finite State Machine |
| ViT | Vision Transformer |
| QAT | Quantization Aware Training |
| PTQ | Post-training Quantization |
| PS | Processing System |
| PL | Programmable Logic |
| AXI | Advanced eXtensible Interface |
| DPHY | Digital Physical Layer |
| GTH | Gigabyte Transceiver High-Speed |
| SRAM | Static Random Access Memory |
| BRAM | Block Random Access Memory |
| LUT | Lookup Table |
| MIPI | Mobile Industry Processor Interface |
| CSI-2 | Camera Serial Interface-2 |
| MIG | Memory Interface Generator |
| ARM | Advanced RISC Machine |
| SoC | System on Chip |
| SoT | Start-of-Transmission |
| ISP | Image Signal Processing |
| DMA | Direct Memory Access |
| PBViT | Push-broom Vision Transformer |
| HFViT | Hardware-friendly Vision Transformer |
| SSFE | Spatial-Spectral Frequency Encoding |
| 1D-FFT | One-dimensional Fast Fourier Transform |
| 1D-IFFT | One-dimensional Inverse Fast Fourier Transform |
| LN | Layer Normalization |
| BN | Batch Normalization |
| PE | Processing Element |
| MAC | Multiply-Accumulate |
| OA | Overall Accuracy |
| AA | Average Accuracy |
| Kappa | Cohen's Kappa |
| Licorice | Glycyrrhiza Uralensis |
| Maca | Lepidium Meyenii |

Spectral data is sourced through the sensor board, as shown in Fig. 1(c). The photodetector employs the SC130GS from SMARTSENS—a 1.3-megapixel global shutter CIS with high quantum efficiency in the 400–700 nm spectral range. The board integrates a crystal oscillator to provide a 27 MHz input clock to the CIS. Since the sensor operates with 1.8 V $I^2C$ logic levels, while the VCCO of the SoC I/O bank is 1.2 V, a PCA9306 level translator is integrated to ensure voltage compatibility for bidirectional communication.

## B. Functionality Description

The general functional block diagram of the spectral camera electronic system is shown in Fig. 2. The CIS on the sensor board transmits the spectral images formed by the optical system to the core board through a four-lane MIPI CSI-2 interface. Meanwhile, the host computer sends commands through the serial port to the FPGA, which controls the CIS configuration registers via the $I^2C$ interface, allowing the host computer to adjust the CIS operating mode.

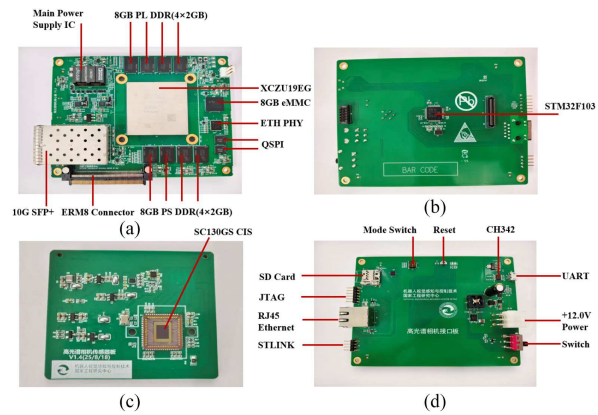


Fig. 1. Photograph of the three boards in the electronic system. (a) Core board. (b) Interface board (front). (c) Sensor board. (d) Interface board (back).

The core board is the central component of the entire system, responsible for spectral reception, spectral preprocessing, on-chip computation for the real-time CHM classification, and high-speed data transmission. In the spectral reception module, the FPGA utilizes DPHY resources to decouple LP and HS signals of the MIPI CSI-2 protocol. IDELAY primitives ensure clock–data phase alignment, followed by serial-to-parallel conversion using SERDES. Byte alignment and lane alignment are then achieved using SoT packets. For spectral preprocessing, the spectral ISP module employs a pipelined architecture for pixel binning, white reference correction, and spectral curve smoothing. For on-chip computation, the PBViT accelerator is integrated and detailed in Section III-C. The system is equipped with 8 GB of DDR4 memory dedicated to PBViT inference, used for buffering preprocessed spectral data. The accelerator's weights are stored on the SD card and then cached in the PS DDR4 before being streamed into the accelerator via AXI DMA. Classification results from the PBViT accelerator are delivered through DMA to the PS DDR4, and then transmitted to the host computer for display via the PS UART interface. High-speed data transmission via 10G SFP+, implemented through GTH transceivers and the UDP protocol stack, streams the ISP-processed spectral images to the host computer. The host computer performs debugging of the ZU19EG through the JTAG interface.

In power control, an STM32F103 MCU monitors the PG outputs of each power chip and issues the corresponding EN signals in a predefined sequence, ensuring reliable system startup. The MCU communicates with the host computer via STLINK for control and monitoring.

# III. FPGA-ORIENTED NETWORK DESIGN AND DEPLOYMENT

This section introduces the PBViT architecture, QAT quantization, and accelerator design.

## A. Network Structure

PBViT consists of two parallel branches: a HFViT branch and a SSFE branch, as shown in Fig. 3. The input to the model

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:17:17 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.



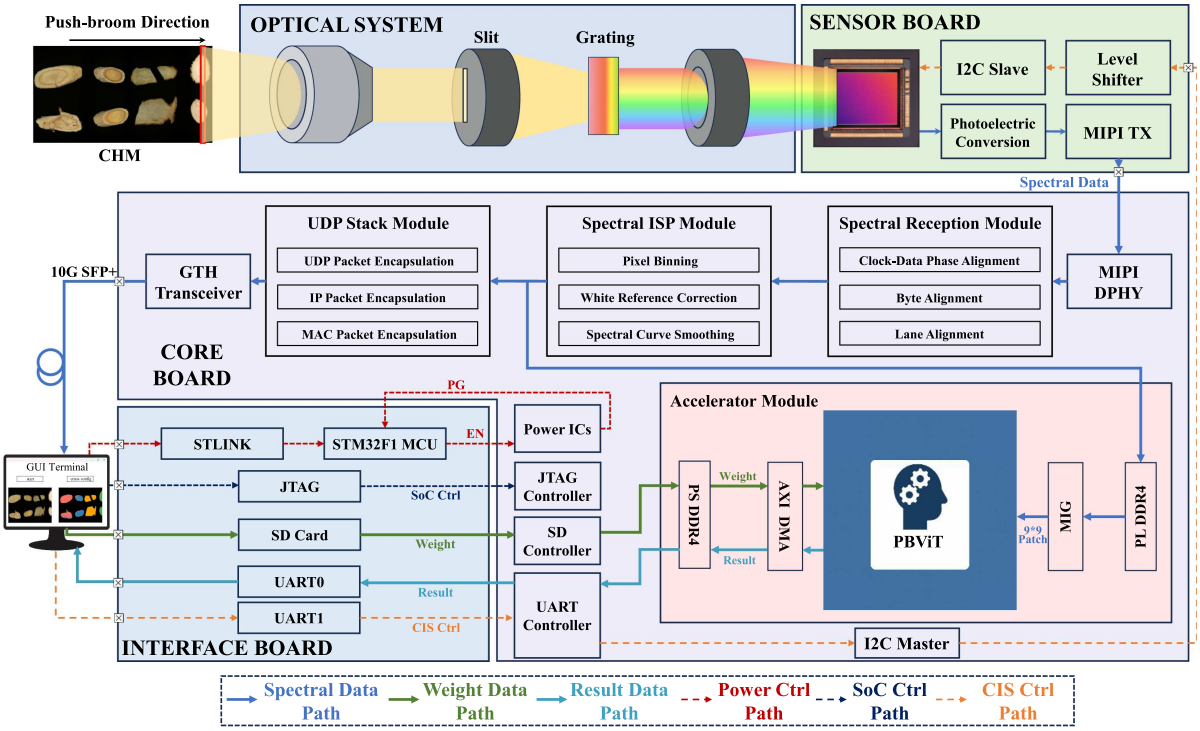


Fig. 2. General functional block diagram of the electronic system. The spectral data from the CIS is preprocessed by the spectral reception and ISP Modules, then transmitted via the UDP stack to the host, while also being buffered in PL DDR4 for PBViT input. Weight data, stored on the SD Card, is loaded into PBViT via AXI DMA. Classification results are sent to the host via UART. Power control is managed by the MCU, monitored through STLINK, while SoC control is via JTAG, and CIS control is handled via I$^2$C, with commands sourced from the host UART.

is a $9 \times 9$ patch, with a spatial dimension of 81 and a spectral dimension of 63.

*1) Spatial–Spectral Frequency Encoding Branch:* HSI contain structured information across both spatial and spectral dimensions, where class-relevant textures and spectral features manifest as specific frequency components. To leverage this property, we introduce a SSFE branch composed of stacked encoding blocks. Given an input patch $\mathbf{F}_{\text{in}} \in \mathbb{R}^{H \times L}$, where $H$ and $L$ are spatial dimensions and spectral channels, each block applies a 1D-FFT along the spatial axis to reveal frequency patterns. A learnable filter $\mathbf{G} \in \mathbb{R}^{\lfloor (H/2) \rfloor + 1}$ modulates the frequency response via element-wise multiplication, followed by a 1D-IFFT to reconstruct the signal. The frequency filtering module is expressed as

$$\mathbf{F}_{\text{out}} = \text{IFFT}\left(\mathbf{G} \odot \text{FFT}(\mathbf{F}_{\text{in}})\right) \tag{1}$$

where $\odot$ denotes element-wise multiplication. The output is then normalized, activated, as follows:

$$\mathbf{F}_{\text{space_out}} = \text{RELU}(\text{BN}(\mathbf{F}_{\text{out}})). \tag{2}$$

Next, the $\mathbf{F}_{\text{space_out}}$ is transposed, and the same operations are performed along the spectral dimension to extract features. Finally, the encoding blocks are concatenated, and residual connections are introduced to enable feature communication across blocks.

While effective on GPU, Fourier transforms introduce latency and resource overhead on FPGA due to complex number operations. To address this, the filter $\mathbf{G}$ is designed with (i) shared frequency response for consistent filtering across the input, and (ii) conjugate symmetry to ensure a real-valued inverse Fourier transform. By the convolution theorem [20], frequency-domain multiplication corresponds to temporal convolution, and with conjugate symmetry, the inverse transform of $\mathbf{G}$ yields a real kernel, enabling the operation to be reformulated as

$$\mathbf{F}_{\text{out}} = \mathbf{C}_{\text{g}}\mathbf{F}_{\text{in}}, \quad \text{with} \quad \mathbf{C}_{\text{g}} = \text{circ}\left(\text{IFFT}(\mathbf{G})\right) \tag{3}$$

where $\text{circ}(\cdot)$ denotes the circulant matrix constructed from the inverse Fourier transform of $\mathbf{G}$. During inference and quantization, we adopt this computation method. This reformulation eliminates the need for dedicated frequency-domain circuit and allows reuse of standard matrix multiplication circuits.

*2. Hardware-Friendly ViT Block:* During inference, LN incurs latency due to real-time computation of mean and variance used in the normalization and scaling [21]. To reduce this overhead, HFViT replaces LN with BN, which leverages precomputed statistics. Empirical results confirm that this substitution maintains classification accuracy. The HFViT can be formalized as

$$\mathbf{Z} = \text{BN}\left(\mathbf{X} + \text{MSA}(\mathbf{X})\right) \tag{4}$$

$$\mathbf{Y} = \text{BN}\left(\mathbf{Z} + \text{FFN}(\mathbf{Z})\right) \tag{5}$$

where $\mathbf{X}$ is the input, $\mathbf{Z}$ the result after attention and normalization, and $\mathbf{Y}$ the output. $\text{MSA}(\cdot)$ denotes multihead self-attention, and $\text{FFN}(\cdot)$ is a two-layer feed-forward network with GELU activation.

*3) Soft Voting-Based Classifier:* Each branch of the model has an independent classification head, consisting of a linear layer that maps encoded features to the output dimensions.

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:17:17 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.



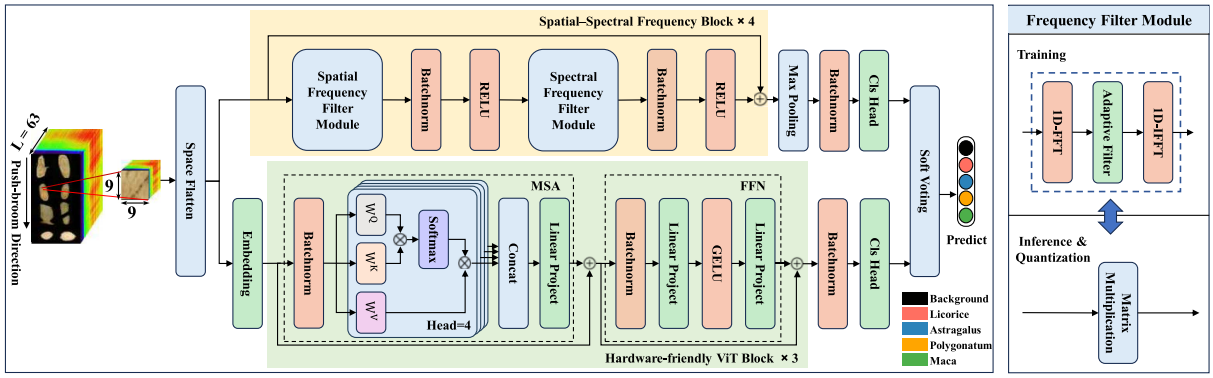


Fig. 3. Overall architecture of PBViT, consisting of two parallel branches: HFViT and SSFE. Final predictions are obtained via soft voting. The frequency filter module is trained using Fourier transforms and deployed via circulant matrix multiplication during quantization and inference.

The final prediction is obtained by aggregating the Softmax probabilities from both branches using a soft voting mechanism, as described by

$$\hat{y} = \text{argmax}\left(\sum_{i=1} \text{Softmax}(\mathbf{v}_i) + \text{Softmax}(\mathbf{s}_i)\right) \tag{6}$$

where $\hat{y}$ is the predicted result, $\mathbf{v}_i$ is the output from the HFViT branch, and $\mathbf{s}_i$ is the output from the SSFE branch.

## B. Network Quantization

To eliminate floating-point operations during deployment, we adopt QAT to train the model under quantization constraints. Unlike PTQ, QAT incorporates quantization noise during retraining, allowing the model to retain accuracy under low-bit inference.

The model's weights and activations are symmetrically and uniformly quantized to integers, allowing all computations to be executed using low-bit arithmetic units. The quantization procedure is defined as

$$x_{\text{int}} = \text{clip}\left(\text{round}\left(\frac{x}{s}\right), -2^{b-1}, 2^{b-1}-1\right) \tag{7}$$

where $x$ denotes the floating-point value, $s$ is the min-max computed scale factor, $b$ is the fixed bit width (set to 8), and $\text{clip}(\cdot)$ enforces the integer bounds.

Quantization granularity plays a key role in balancing computational efficiency and representational fidelity. Finer granularity allows for more precise control over the model's representation and can potentially improve classification accuracy. However, it often increases memory overhead, as a large number of quantization parameters need to be prestored, and also increases data transfer bandwidth. On the other hand, coarser quantization simplifies hardware implementation by reducing resource demands and improving deployment efficiency on hardware accelerators. To balance efficiency and accuracy, we quantize input activations in a token-wise manner to preserve fine-grained spatial-spectral differences. Subsequent activations adopt layer-wise quantization for improved hardware compatibility, and all weights are quantized channel-wise to maintain neural network expressive capacity.

Further improvements in inference efficiency are achieved by systematically fusing nonlinear element-wise operations with adjacent layers. BN and quantization, both element-wise, are consolidated into unified computation units. In linear layers, bias addition and quantization are also integrated immediately after matrix multiplication, reducing redundant operations. Additionally, for the nonlinear functions Softmax and GELU, we employ second-order polynomial approximation [22], as detailed in Section III-C3. This method converts the computation into standard arithmetic operations, allowing for deployment using a unified hardware circuit.

## C. Network Accelerator

An overlapping execution paradigm is adopted to maximize resource reuse under the logic and memory constraints of the FPGA, while isolating the control and computation circuits to enhance the scalability of the accelerator.

As illustrated in Fig. 4, the PBViT accelerator integrates three types of compute modules: two linear cores, two nonlinear cores, and two residual cores. These compute modules are supported by on-chip SRAMs for intermediate data buffering, DDR4 memory (via MIG) for streaming line-scan inputs, and a DMA interface for dynamically loading model weights, all managed by the central controller.

*1) Temporally Overlapped Scheduling for the SSFE Branch:* During the execution of the attention mechanism in the HFViT block, the Softmax operation involves three sequential steps: max-value extraction, exponentiation, and normalization. The first two stages each consume a full dataflow cycle, while the normalization step, implemented via a Radix-2 division algorithm on FPGA, incurs a latency of approximately 28 clock cycles. This results in a idle window for the linear core. To maximize hardware utilization, we schedule the circulant matrix multiplication from the SSFE branch to execute within this window, hiding its inference latency. Furthermore, the output of the SSFE computation requires BN processing, which is handled by the nonlinear core. However, the first public DSP of this core is currently engaged in Softmax computation. To avoid contention, we repurpose the private DSP from the GELU pathway—unused during Softmax—to execute the BN operation.

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:17:17 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.



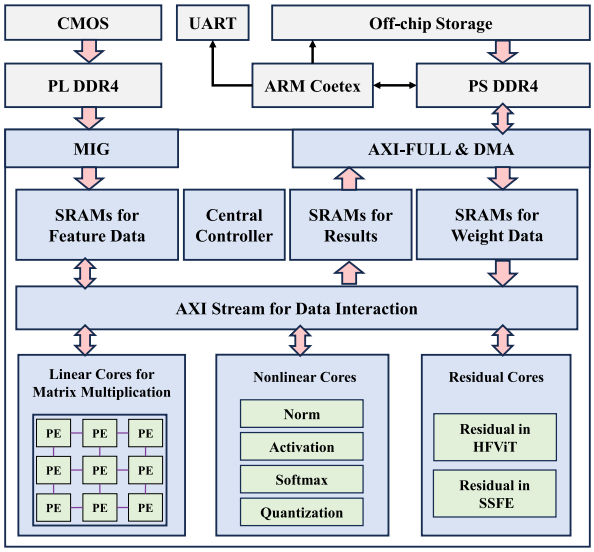


Fig. 4. Overall architecture of the PBViT accelerator comprises linear, nonlinear, and residual cores, along with other components. On-chip SRAMs, PL DDR4 via MIG, and DMA interfaces enable high-throughput dataflow scheduling coordinated by the central controller.

*2) Linear Core:* The linear cores handle all matrix multiplications in the model. Conventional multiplication architectures, such as weight-stationary (WS) [23], output-stationary (OS) [24], and input-stationary (IS) [25], require reshaping and staging of input tensors to match dataflow patterns, increasing control complexity. To address this, we design linear cores based on an outer-product architecture that streams one column of matrix A and one row of matrix B per cycle into the PE array, as illustrated in Fig. 5(a), computing and accumulating the corresponding outer product without data reformatting and intermediate buffering. Each linear core is built using a $P \times P$ (set to 32) systolic-like PE array and performs partial matrix multiplication based on the outer-product formulation. Given input matrices $A \in \mathbb{R}^{M\times K}$ and $B \in \mathbb{R}^{K\times N}$, the product $C = AB \in \mathbb{R}^{M\times N}$ is computed as

$$C^{(m,n)} = \sum_{k=1}^{K} A_{:,k}^{(m)} \cdot B_{k,:}^{(n)} \tag{8}$$

where $A_{:,k}^{(m)} \in \mathbb{R}^{P\times 1}$ and $B_{k,:}^{(n)} \in \mathbb{R}^{1\times P}$ are column and row partitions of $A$ and $B$, respectively. When $M$ or $N$ exceeds $P$, the matrix is divided into blocks and computed across multiple linear cores. The final result $C$ is assembled as

$$C = \bigoplus_{m=1}^{M/P} \bigoplus_{n=1}^{N/P} C^{(m,n)}. \tag{9}$$

Since each sub-block $C^{(m,n)}$ is computed independently, the proposed structure facilitates parallel execution without requiring inter-core communication.

The architecture of PE is shown in Fig. 5(b). Each PE comprises MAC logic, registers, auxiliary control circuitry, and routing units. PEs perform pipelined MAC operations and store results in internal registers. Multiplications are mapped to DSP48E2 blocks, which support $27 \times 18$-bit signed arithmetic. To optimize utilization, a DSP packing strategy is adopted: two 8-bit operands from vertically adjacent PEs are shift-added and multiplied with a shared 8-bit operand via the DSP's preadder path. All operands are sign-extended to prevent overflow and cross-term interference, enabling each DSP to execute two 8-bit multiplications in parallel, reducing DSP usage by 50%. To support data transpose, each PE can forward its output to either the east or south neighbor, enabling results to propagate horizontally or vertically.

*3) Nonlinear Core:* The nonlinear cores implement a unified module supporting quantization, BN, Softmax, and GELU. Each core consists of 32 nonlinear PEs, with these functions time-multiplexed. Each PE contains four DSPs: the first is public, the next two are shared between Softmax and GELU, and the final one is shared between GELU in the HFViT and BN in the SSFE branch.

*a) Fused BatchNorm and quantization:* Both BN and quantization are element-wise operations during inference, with all required parameters precomputed per channel. This allows them to be fused into a single computational pass. Specifically, the BN operation is shown as follows:

$$\mathbf{Y} = \boldsymbol{\gamma} \odot \frac{\mathrm{s}_1 \cdot \mathbf{X} - \boldsymbol{\mu}}{\sqrt{\boldsymbol{\sigma}^2 + \epsilon}} + \boldsymbol{\beta} \tag{10}$$

where $\mathbf{X}$ is the integer input, $\mathrm{s}_1$ is the scale factor from the previous step, and $\boldsymbol{\gamma}$, $\boldsymbol{\beta}$, $\boldsymbol{\mu}$, $\boldsymbol{\sigma}^2$ are per-channel vectors representing scale, bias, mean, and variance, respectively; $\epsilon$ is a small constant for numerical stability.

The quantization step maps the normalized outputs to integers

$$\mathbf{Z}_{\text{int}} = \text{clip}\left(\text{round}\left(\frac{\mathbf{Y}}{\mathrm{s}_2}\right), -2^{b-1}, 2^{b-1} - 1\right) \tag{11}$$

where $\mathrm{s}_2$ is the quantization scale for the result, and $b$ is the quantization bit width. By fusing both steps, the final expression becomes

$$\mathbf{Z}_{\text{int}} = \text{clip}\left(\text{round}\left(\mathbf{X} \odot \mathbf{B} + \mathbf{C}\right), -2^{b-1}, 2^{b-1} - 1\right) \tag{12}$$

where

$$\mathbf{B} = \frac{\boldsymbol{\gamma} \cdot \mathrm{s}_1}{\sqrt{\boldsymbol{\sigma}^2 + \epsilon} \cdot \mathrm{s}_2}, \quad \mathbf{C} = \frac{\boldsymbol{\beta} - \boldsymbol{\gamma} \odot \boldsymbol{\mu}/\sqrt{\boldsymbol{\sigma}^2 + \epsilon}}{\mathrm{s}_2}. \tag{13}$$

These coefficients are stored as 16-bit fixed-point values in BRAM-based ROMs, enabling each fused operation to use a single public DSP via the $A \times B + C$ arithmetic path.

*b) Fused linear bias and quantization:* In the linear layer, bias addition and output quantization are fused into a single operation following matrix multiplication. The scaling terms are computed as

$$\mathbf{B} = \frac{\mathbf{s}_{\text{in}} \otimes \mathbf{s}_{\text{fc}}}{\mathbf{s}_{\text{out}}}, \quad \mathbf{C} = \frac{\mathbf{bias}}{\mathbf{s}_{\text{out}}} \tag{14}$$

where $\mathbf{s}_{\text{in}}$, $\mathbf{s}_{\text{fc}}$, and $\mathbf{s}_{\text{out}}$ denote the scale factors of the input, weights, and output. The fused quantization is expressed as

$$\mathbf{Y}_{\text{int}} = \text{clip}\left(\text{round}\left(\mathbf{S} \odot \mathbf{B} + \mathbf{C}\right), -2^{b-1}, 2^{b-1} - 1\right) \tag{15}$$

where $\mathbf{S}$ is the result of the linear layer's matrix multiplication. This affine formulation uses only one public DSP.

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:17:17 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.



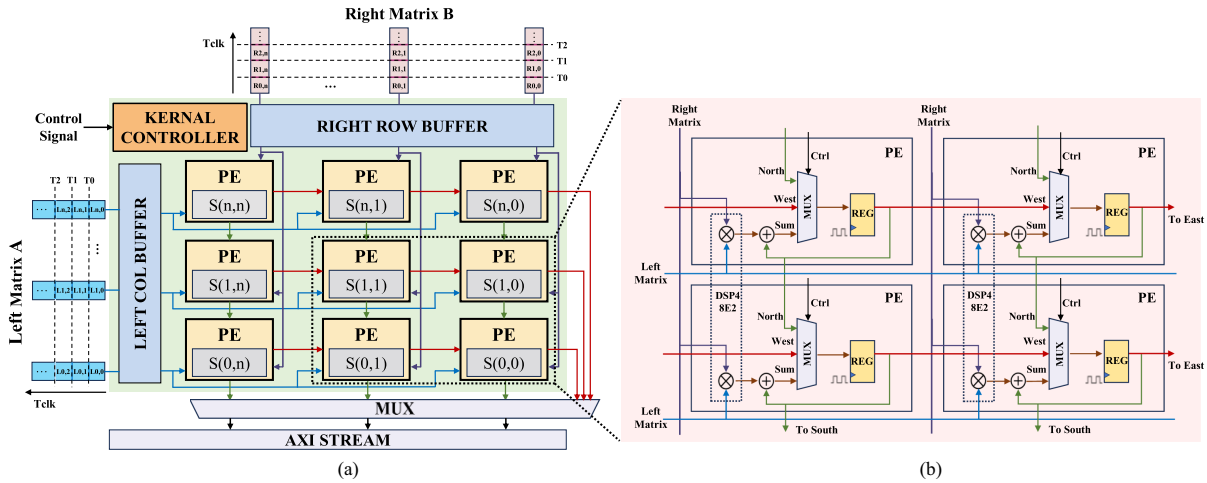


Fig. 5. (a) Overall architecture of the linear core. Left matrix columns and right matrix rows are streamed into the core for pipelined matrix multiplication. The output is selected via a multiplexer to transmit either row-wise or column-wise. (b) PE circuit architecture. Each vertical pair of PEs shares a single DSP block. Under control instructions, PE outputs are propagated eastward or southward as needed.

*c) Integer-only softmax and GELU:* We adopt the approach of [22] to transform both Softmax and GELU activations into integer operations. The Softmax function is defined as

$$\mathrm{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} = \frac{e^{x_i - x_{\max}}}{\sum_j e^{x_j - x_{\max}}} \tag{16}$$

where $x_{\max} = \max_j x_j$, the exponential term is then decomposed as

$$e^{\Delta x} = 2^{-q_i} e^{r_i}, \ \Delta x = x_i - x_{\max}, \ e^{r_i} \approx a r_i^2 + b r_i + c \tag{17}$$

where $q_i = \lfloor \Delta x / (-ln2) \rfloor$ is the integer exponent, $r_i = \Delta x + q_i * ln2$ is the residual, and $a$, $b$, $c$ are fixed quadratic approximation coefficients stored in registers. Therefore, to compute Softmax, a comparator identifies the maximum value, and a subtractor computes $\Delta x_i$. Three DSPs are employed to calculate $q_i$, $r_i$, and the second-order polynomial. The first DSP is share with the public, while the other two are shared with GELU. Finally, the result is obtained using a divider implementing a Radix-2 division algorithm based on LUTs, with a latency of 28 clock cycles.

Similarly, the GELU activation is expressed as

$$\mathrm{GELU}(x) = x \cdot \frac{1}{2}\left(1 + \mathrm{erf}\left(\frac{x}{\sqrt{2}}\right)\right) \tag{18}$$

where $\mathrm{erf}(\cdot)$ denotes the Gaussian error function. To enable integer-only execution, we approximate the error function using a signed quadratic form

$$\mathrm{erf}\left(\frac{x}{\sqrt{2}}\right) \approx \mathrm{sign}(x) \cdot (a|x|^2 + b|x| + c). \tag{19}$$

Therefore, computing GELU requires four DSPs: the first for quantizing the output of the linear layer before GELU, the second for the quadratic approximation of the error function, the third for the final GELU expression, and the fourth for quantizing the final result. The first DSP is a public DSP, the next two are shared with Softmax, and the last with BN in the SSFE branch.

*4) Residual Core and Central Controller:* The residual computation cores are used to calculate the skip connections within each branch. Each core is composed of 32 PEs. The operation is defined by

$$\mathbf{Y} = \mathbf{X}_1 \cdot \frac{s_1}{s_3} + \mathbf{X}_2 \cdot \frac{s_2}{s_3} \tag{20}$$

where $s_1$, $s_2$, and $s_3$ are the quantization factors for $\mathbf{X}_1$, $\mathbf{X}_2$, and $\mathbf{Y}$, respectively. Since the bit width of this operation is low, we utilize LUT to implement the multiplication (using Booth's algorithm) and addition.

The central controller coordinates dataflow and memory transactions across modules, enabling ordered execution and improving overall hardware utilization. The controller is designed using a FSM, with the network depth as a parameter to adjust the state machine's loop depth, facilitating the deployment of models with varying depths. To clearly demonstrate the scheduling of the controller and the accelerator's latency, the RTL simulation is shown in Fig. 6.

## IV. Experiments and Results

To validate the effectiveness of the proposed Algorithm-FPGA co-design, Section IV-A evaluates algorithm performance, while Section IV-B assesses hardware efficiency.

### A. PBViT Algorithm Evaluation

*1) Training Details and Computational Platform:* Our proposed PBViT was trained on the CHM dataset using PyTorch 2.3 in the PyCharm environment, with a batch size of 1280, a learning rate of $5 \times 10^{-4}$, and 1000 epochs. For the Indian Pines and Salinas datasets, a batch size of 128, a learning rate of $1 \times 10^{-3}$, and 500 epochs were used. The Adam optimizer with cross-entropy loss was applied. For QAT quantization, we used a batch size of 1280, a learning rate of $1 \times 10^{-6}$, and 200 epochs, employing SGD as the optimizer. All experiments were conducted on a Windows 11 system with an Intel i9-13900HX CPU and an NVIDIA RTX 4090 Laptop GPU.

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:17:17 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.



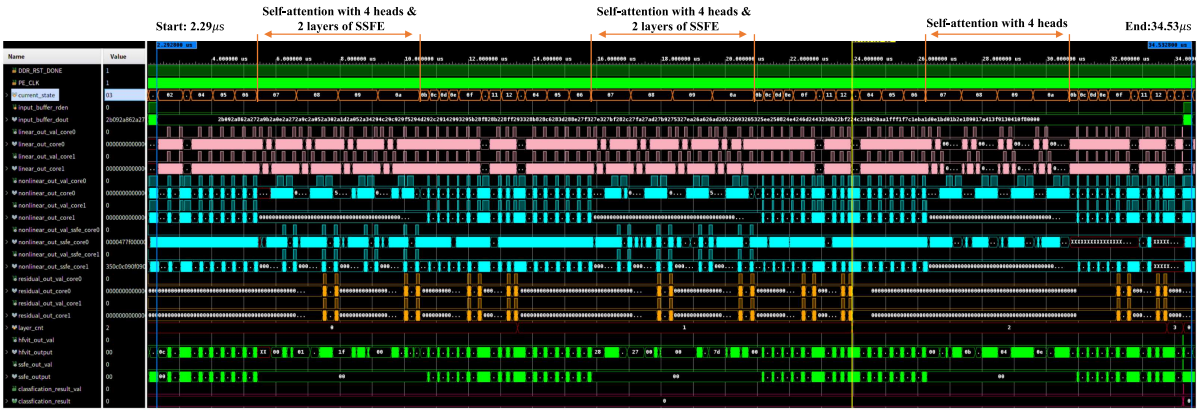


Fig. 6. RTL simulation of the accelerator illustrates the complete inference process for a $9 \times 9$ patch, taking 32.2 $\mu$s. The figure shows key signal lines to observe the central controller, three types of compute cores, and final result, with all signals valid when their corresponding value is high. The temporally overlapped scheduling of the SSFE branch is demonstrated, embedding two layers of SSFE into each 4-head attention inference.

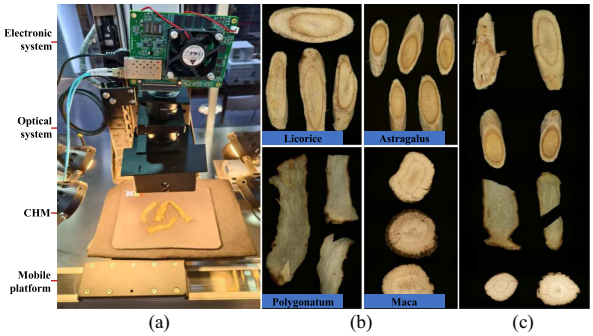


Fig. 7. Spectral imaging system and CHM dataset. (a) Spectral imaging system. (b) Training dataset. (c) Test dataset.

TABLE II
COMPARISON WITH GENERAL HSI CLASSIFICATION MODELS ON CHM DATASET

| Model | OA | AA | Kappa | Param |
|---|---|---|---|---|
| SVM [26] | 93.37±0.13 | 79.95±1.32 | 84.41±0.63 | – |
| RF [27] | 94.32±0.15 | 82.63±0.74 | 87.23±0.36 | – |
| KNN [28] | 92.53±0.09 | 78.91±0.66 | 83.58±0.32 | – |
| 1DCNN [12] | 96.32±0.12 | 90.74±0.92 | 92.07±0.41 | 0.221 M |
| 2DCNN [29] | 96.49±0.07 | 91.46±0.74 | 92.41±0.35 | 0.158 M |
| 3DCNN [30] | 97.27±0.07 | 93.01±0.59 | 93.56±0.30 | 7.846 M |
| Spectralformer [14] | 95.65±0.04 | 89.47±0.43 | 90.90±0.27 | 0.139 M |
| SSFTT [15] | 97.38±0.08 | 93.17±0.59 | 94.13±0.29 | 0.579 M |
| GSCViT [16] | 96.92±0.11 | 92.60±0.81 | 93.25±0.38 | 0.195 M |
| PBViT | 97.81±0.05 | 94.19±0.39 | 94.81±0.27 | 0.075 M |

*2) Dataset Description:* The CHM dataset includes four CHMs: Licorice, Astragalus membranaceus, Polygonatum odoratum, and Maca. The spectral imaging system is illustrated in Fig. 7(a). CHMs are placed on a mobile platform and illuminated by halogen lamps. Reflected light passes through an entrance slit and a grating. The CIS captures one spatial line at a time within the 400–700 nm spectral range, progressively forming a complete hyperspectral cube through push-broom imaging as the platform moves forward. To avoid interference from external light, the imaging device is isolated from the outside by dark-colored glass.

The four CHMs are imaged separately [as shown in Fig. 7(b)], with random selection of $9 \times 9$ patches from each image to form the training set. The training set consists of 500 background samples and 1200 samples each for Licorice, Astragalus, Polygonatum, and Maca, resulting in a total of 5300 samples. To avoid data leakage, the test set is created separately by imaging four CHMs together [as shown in Fig. 7(c)], using all nonoverlapping patches. This results in a total of 31920 test samples: 23480 background, 2778 Licorice, 1650 Astragalus, 2620 Polygonatum, and 1392 Maca.

TABLE III
COMPARISON WITH GENERAL HSI CLASSIFICATION MODELS ON INDIAN PINES DATASET

| Model | OA | AA | KAPPA |
|---|---|---|---|
| 1DCNN [12] | 95.46±0.65 | 93.51±1.70 | 94.82±0.74 |
| 2DCNN [29] | 97.78±0.27 | 97.00±0.49 | 97.47±0.31 |
| 3DCNN [30] | 93.41±1.52 | 86.47±3.64 | 92.45±1.75 |
| Spectralformer [14] | 97.88±0.17 | 95.46±0.76 | 97.58±0.19 |
| SSFTT [15] | 97.34±0.33 | 95.80±0.69 | 96.97±0.38 |
| GSCViT [16] | 98.29±0.13 | **97.08±0.76** | 98.04±0.15 |
| PBViT | 98.32±0.21 | 97.02±0.74 | 98.11±0.24 |

Note: Bold entries indicate the best performance for the corresponding metric within each table.

To evaluate the generalization of PBViT, we selected two publicly available datasets from remote sensing and agriculture: Indian Pines and Salinas. The Indian Pines dataset consists of $145 \times 145$ pixels and 224 spectral bands covering the wavelength range from 400 to 2500 nm, with 16 classes. The Salinas dataset contains $512 \times 217$ pixels and 224 spectral bands, also with 16 classes. We randomly selected 10% of the points from Indian Pines and 1% from Salinas for training, with the remaining points used for testing.

*3) Comparative Experiments:* On the CHM dataset, we benchmark PBViT against classical machine learning models, CNN networks, and ViT-based architectures, using full-precision floating point, as summarized in Table II. To further evaluate PBViT's performance, we provide the confusion matrix in Fig. 8, offering a detailed view of the model's classification accuracy. Additionally, we conducted comparative experiments on the Indian Pines and Salinas datasets, as detailed in Tables III and IV. All experiments were independently repeated 10 times, and the metrics are reported as the mean and standard

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:17:17 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.



TABLE IV
COMPARISON WITH GENERAL HSI CLASSIFICATION MODELS ON SALINAS DATASET

| Model | OA | AA | KAPPA |
|---|---|---|---|
| 1DCNN [12] | 94.60±0.71 | 96.04±0.50 | 93.99±0.78 |
| 2DCNN [29] | 96.22±0.41 | 97.91±0.33 | 95.80±0.45 |
| 3DCNN [30] | 95.26±0.30 | 96.59±0.42 | 94.72±0.33 |
| Spectral former [14] | 98.08±0.25 | 98.60±0.28 | 97.86±0.27 |
| SSFTT [15] | 96.93±0.21 | 97.96±0.30 | 96.58±0.23 |
| GSCViT [16] | 97.02±0.47 | 97.70±0.45 | 96.69±0.52 |
| PBViT | 98.17±0.27 | 98.86±0.12 | 97.96±0.30 |

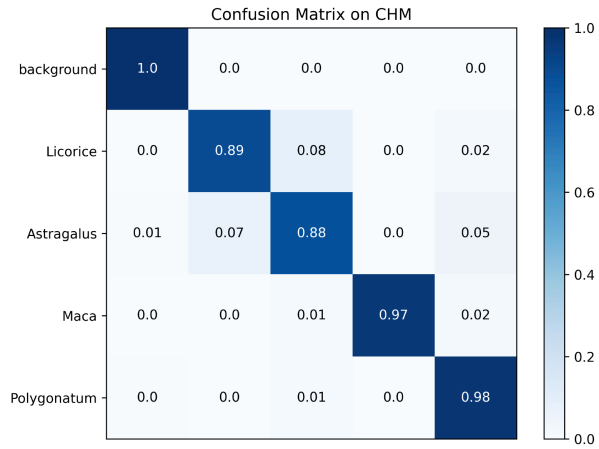


Fig. 8. Confusion matrix of PBViT on the CHM dataset.

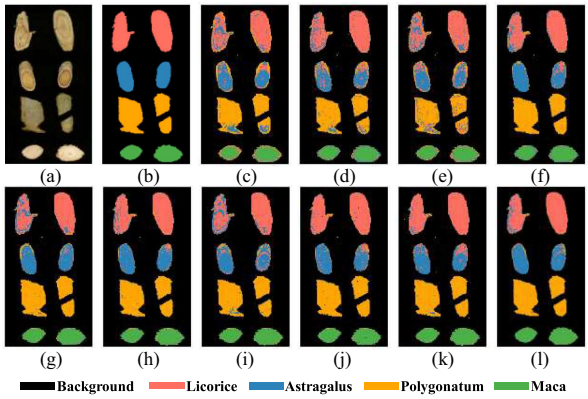


Fig. 9. Visualization of results on CHM dataset, inference is performed using nonoverlapping patches. (a) Original. (b) GT. (c) SVM. (d) RF. (e) KNN. (f) 1DCNN. (g) 2DCNN. (h) 3DCNN. (i) Spectralformer. (j) SSFTT. (k) GSCViT. (l) PBViT.

deviation. Visualizations of the results are shown in Figs. 9–11, respectively.

Machine learning models exhibit limited ability to capture spatial-spectral dependencies, leading to suboptimal classification performance. CNN models show improved results, with 3DCNN achieving notably higher accuracy by jointly modeling spatial and spectral dimensions on the CHM dataset. However, its high parameter count and reliance on complex data reshaping hinder efficient deployment on hardware platforms. ViT-based models leverage global modeling capabilities, achieving top-tier performance across all datasets. However, hybrid architectures, such as SSFTT and GSCViT, introduce irregular

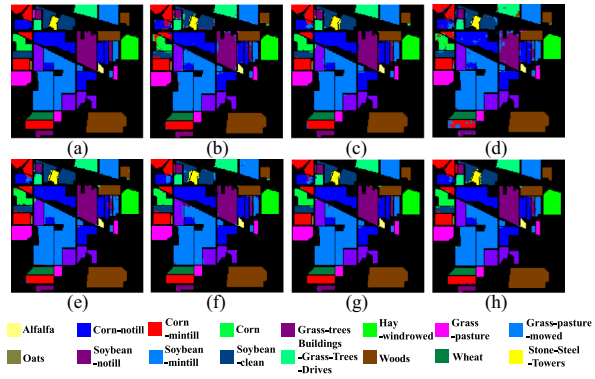


Fig. 10. Visualization of results on Indian Pines dataset. (a) GT. (b) 1DCNN. (c) 2DCNN. (d) 3DCNN. (e) Spectralformer. (f) SSFTT. (g) GSCViT. (h) PBViT.

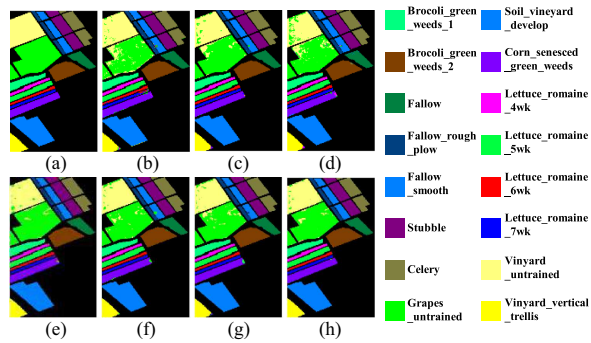


Fig. 11. Visualization of results on Salinas dataset. (a) GT. (b) 1DCNN. (c) 2DCNN. (d) 3DCNN. (e) Spectralformer. (f) SSFTT. (g) GSCViT. (h) PBViT.

TABLE V
ABLATION AND QUANTIZATION EXPERIMENTS ON CHM DATASET

| Model | OA | AA | Kappa |
|---|---|---|---|
| ViT | 0.965 | 0.921 | 0.922 |
| HFViT | 0.965 | 0.914 | 0.921 |
| SSFE | 0.972 | 0.916 | 0.937 |
| HFViT* | 0.966 | 0.915 | 0.922 |
| SSFE* | 0.970 | 0.911 | 0.932 |
| HFViT+SSFE | **0.978** | **0.942** | **0.950** |
| HFViT*+SSFE* | 0.977 | 0.939 | 0.948 |

Note: *Indicates quantized inference with 8-bit weights and activations. Bold entries indicate the best performance for the corresponding metric within each table.

topologies that complicate accelerator design. PBViT achieves the highest scores with low standard deviation while maintaining the lowest parameter count and a regular, hardware-friendly structure.

*4) Ablation Study and Quantization Evaluation:* To assess the individual impact of each proposed component, we conduct ablation studies on the CHM dataset. As summarized in Table V, substituting LN in ViT with BN (HFViT) yields comparable performance, indicating that BN serves as a hardware-friendly alternative without degrading accuracy. Incorporating the SSFE branch alone—despite introducing only 0.002 M parameters—achieves 0.972 OA, 0.916 AA, and 0.937 Kappa, underscoring the effectiveness of frequency-domain modeling

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:17:17 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.



TABLE VI
QUANTIZATION PRECISION OF PTQ AND QAT UNDER DIFFERENT BIT WIDTHS

| Method | 4-bit | | | 6-bit | | | 8-bit | | | 10-bit | | | 12-bit | | | 14-bit | | | 16-bit | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | OA | AA | Kappa | OA | AA | Kappa | OA | AA | Kappa | OA | AA | Kappa | OA | AA | Kappa | OA | AA | Kappa | OA | AA | Kappa |
| PTQ | 81.93 | 47.54 | 59.56 | 94.88 | 88.34 | 89.74 | 97.18 | 92.91 | 93.63 | 97.65 | 93.83 | 94.68 | 97.68 | 93.82 | 94.75 | 97.74 | 93.84 | 94.88 | 97.73 | 93.95 | 94.87 |
| QAT | 88.55 | 58.93 | 73.98 | 96.08 | 90.52 | 91.21 | 97.73 | 93.91 | 94.82 | 97.70 | 94.07 | 94.81 | 97.66 | 94.14 | 94.70 | 97.74 | 93.85 | 94.88 | 97.76 | 94.01 | 94.93 |

TABLE VII
RESOURCE USAGE ON THE FPGA IMPLEMENTATION OF PROCESSOR

| FPGA Resource | Used | Available | Utilization |
|---|---|---|---|
| LUT | 195072 | 522720 | 37.32% |
| LUTRAM | 8167 | 161280 | 5.06% |
| FF | 202729 | 1045440 | 19.39% |
| BRAM36 | 153 | 984 | 15.55% |
| DSP | 1284 | 1968 | 65.24% |
| GT | 1 | 48 | 2.08% |

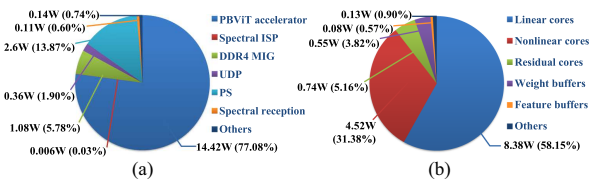


Fig. 12. (a) Processor dynamic power consumption breakdown. (b) PBViT accelerator dynamic power consumption breakdown.

in HSI analysis. When HFViT and SSFE are jointly applied, the model achieves the highest overall performance, with notable improvements in AA (from 0.921 to 0.942) and Kappa (from 0.922 to 0.950), demonstrating the strong synergy between HFViT and SSFE branch.

To evaluate the effect of quantization, 8-bit integer inference is applied to both HFViT and SSFE, denoted as HFViT* and SSFE*. Individually quantized modules exhibit negligible performance degradation. When both components are jointly quantized, AA and Kappa decrease by only 0.3% and 0.2%, respectively, compared with the full-precision model.

Furthermore, we compared the quantization precision of PTQ and QAT under different bit widths, as shown in Table VI. It can be observed that for higher bit widths, both PTQ and QAT exhibit negligible precision degradation. When quantized to 8-bit, however, PTQ suffers from a noticeable drop in precision due to the reduced representation range of weights and activations, while QAT enables the network to adapt to quantization errors through online training, resulting in an almost lossless performance. For lower bit widths, both PTQ and QAT show significant precision degradation, as the input spectral curves inherently exhibit small variations, and the characteristic peaks—crucial for classification—are completely overwhelmed by quantization noise. Therefore, our system adopts 8-bit QAT quantization, which effectively reduces resource consumption while maintaining high precision.

## B. Hardware Evaluation

*1) Resource Utilization and Power Analysis:* The resource utilization of the proposed integrated sensing–computing processor, implemented in Verilog on the Zynq UltraScale+ ZU19EG platform, is summarized in Table VII. The overall

TABLE VIII
ABLATION RESULTS OF ACCELERATOR IMPLEMENTATION

| Model | Resource | | | | Metric | |
|---|---|---|---|---|---|---|
| | LUT | FF | BRAM36 | DSP | Latency ($\mu$s) | Throughput (GOPS) |
| HFViT | 165301 | 163716 | 89 | 1280 | **31.1** | 385 |
| PBViT | 172135 | 174972 | 112 | 1280 | 32.2 | **502** |

Note: Bold entries indicate the best performance for the corresponding metric within each table.

LUT utilization reaches 37.32%, primarily attributed to the overlapping execution paradigm, which introduces extensive multiplexer logic, as well as to the Radix-2 divider used for Softmax normalization and the 8-bit MAC units in the residual cores—both implemented with LUTs. BRAM consumption is dominated by buffering of intermediate features and model weights; due to the wide data path, 153 blocks of 36 K BRAMs are utilized. The PBViT accelerator incorporates two $32 \times 32$ linear cores, which together require 1024 DSPs through packing optimization. Additionally, two nonlinear cores are implemented, each consisting of 32 PEs, with each PE requiring four DSPs. The DDR4 MIG controller also consumes DSPs for address computation. In total, 1284 DSPs are used, accounting for 65.24% of the available DSP resources.

Power analysis is performed using postimplementation reports from Vivado 2020.2 to evaluate power consumption across processor modules. With the PBViT accelerator operating at 300 MHz, the total processor power consumption reaches 20.15 W, comprising 18.71 W of dynamic power and 1.45 W of static power. The PBViT accelerator accounts for 14.42 W of dynamic power—corresponding to 77.08% of the total—shown in Fig. 12(a). A breakdown of internal power consumption is shown in Fig. 12(b). Linear cores dominate at 58.15% due to densely packed MAC units for pipelined matrix computation. Nonlinear cores contribute 31.38%, executing nonlinear functions in the quantized inference pipeline.

*2) Validation of the Overlapped Scheduling:* The temporally overlapped scheduling strategy for the SSFE branch is evaluated in terms of resource utilization and inference latency. As detailed in Section III-C1, SSFE reuses the ViT branch's linear and nonlinear compute cores, requiring only additional control logic, multiplexers, and cache. As reported in Table VIII, this design introduces only marginal overhead: LUTs increase from 165301 to 172135 (+4.13%), FFs from 163716 to 174972 (+6.88%), and BRAMs from 89 to 112 (+25.84%), with no additional DSP usage. Inference latency increases by only 1.1 $\mu$s, while throughput improves by 30.4%. These results confirm that the SSFE branch incurs minimal hardware and latency costs, yet delivers notable gains in classification performance.

*3) Comparison With Existing Accelerators:* We compare our proposed accelerator with existing FPGA-based designs to validate its performance, as summarized in Table IX. Due to

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:17:17 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.



TABLE IX
COMPARISON WITH EXISTING FPGA ACCELERATORS

| | [18] | [18] | [31] | [32] | [33] | Ours |
|---|---|---|---|---|---|---|
| FPGA | ZU15EG | ZU15EG | XCU50 | ZU9EG | XCU250 | ZU19EG |
| Model | HybridSN | C3D | Swin-T | DeiT-T | DeiT-S | PBViT |
| Input Type | his | HSI | RGB | RGB | RGB | HSI |
| Frequency (MHz) | 100 | 100 | 300 | 150 | 300 | 300 |
| LUT (K) | 256 | – | 258 | 138 | 798 | 172 |
| FF (K) | 103 | – | 257 | 126 | – | 175 |
| BRAM36 | 600 | – | 1002 | 355.5 | 960 | 112 |
| DSP | 1307 | 1307 | 2420 | 1968 | 7088 | 1280 |
| Power (W) | – | 8.4 | 39 | 9.45 | – | 14.42 |
| Throughput (GOPS) | 27 | 160 | 309.6 | 203 | 1800 | 502 |
| Power Efficiency (GOPS/W) | – | 19.08 | 7.94 | 21.48 | – | **34.81** |

Note: Bold entries indicate the best performance for the corresponding metric within each table.

TABLE X
CROSS-PLATFORM INFERENCE SPEED AND POWER COMPARISON

| Architecture | Device | Technology (nm) | Frequency (MHz) | Throughput (GOPS) | Power (W) | Power Efficiency (GOPS/W) |
|---|---|---|---|---|---|---|
| CPU | i9-13900HX | 10 | 3750 | 71 | 128.5 | 0.55 |
| GPU | RTX 4090 Laptop | 4 | 2225 | **661** | 118.4 | 5.58 |
| FPGA | Xilinx ZU19EG | 16 | 300 | 502 | **14.42** | **34.81** |

Note: Bold entries indicate the best performance for the corresponding metric within each table.

the scarcity of accelerators designed for HSI classification, we include [18] as a representative work, which presents a convolutional architecture for this task. However, the use of 3-D convolutions imposes strict constraints on dataflow, requiring off-chip memory accesses and data reshaping by the ARM processor. These overheads degrade throughput, yielding only 27 GOPS for the HybridSN and 160 GOPS for the C3D.

We also compared three representative ViT accelerators on FPGA, all of which use RGB images as input. Wang et al. [31] designed an accelerator for the Swin Transformer, which operates in floating point without quantization, leading to high power consumption and substantial resource utilization. Dong et al. [32] employs a pruning strategy to compress the model, but the pruning reduces the number of operations, and with a clock speed of 150 MHz, the throughput is limited to 203 GOPS. Parikh et al. [33] achieves the highest throughput, however, its excessive resource consumption and expensive deployment chip make it unsuitable for edge applications. In contrast, our accelerator strikes the best power-performance balance, delivering 502 GOPS throughput and 34.81 GOPS/W energy efficiency, enabling real-time inference that keeps pace with the CIS spectral image collection.

*4) Cross-Platform Inference Comparison:* Inference speed and power are compared across an i9-13900HX CPU, an RTX 4090 Laptop GPU, and the proposed accelerator on ZU19EG. Results are summarized in Table X. Due to limited parallelism, the CPU exhibits the slowest speed. The GPU, equipped with massive compute cores, achieves the fastest inference but at the cost of high power consumption. In contrast, the FPGA achieves a balanced tradeoff between speed and power: its inference time is 75.95% of the GPU's while consuming only 14.42 W—leading to energy efficiency improvements of 6.24× and 63× over the GPU and CPU, respectively.

## V. CONCLUSION

This work presents an integrated sensing–computing processor for real-time HSI classification. The processor combines a push-broom spectral imaging system, a hardware-efficient dual-branch network, PBViT, optimized for FPGA deployment, and a high-performance acceleration architecture. An electronic system was designed as the hardware foundation for the processor. PBViT leverages the complementary strengths of the SSFE and HFViT branches to achieve superior classification performance. We use QAT to quantize the model to 8-bit integers for FPGA optimization. The accelerator adopts an overlapping execution paradigm, with three types of computation cores and cache completing network calculations under FSM scheduling, achieving ideal throughput and energy efficiency. In conclusion, this processor offers a novel, hardware-efficient solution for edge-based spectral detection, demonstrating strong applicability in the pharmaceutical industry. Its flexibility allows for easy adaptation to various industrial scenarios, while its ability to meet regulatory and industrial requirements ensures its broad impact.

## REFERENCES


[1] Q. Wang, Q. Li, and X. Li, "Hyperspectral image superresolution using spectrum and feature context," *IEEE Trans. Ind. Electron.*, vol. 68, no. 11, pp. 11276–11285, Nov. 2021.

[2] H. Zhang, X. Jiang, L. Liu, H. Wang, and Y. Wang, "S2F2AN: Spatial–spectral fusion frequency attention network for Chinese herbal medicines hyperspectral image segmentation," *IEEE Trans. Instrum. Meas.*, vol. 74, pp. 1–13, 2025.

[3] H. Radner, J. Stange, L. Büttner, and J. Czarske, "Field-programmable system-on-chip-based control system for real-time distortion correction in optical imaging," *IEEE Trans. Ind. Electron.*, vol. 68, no. 4, pp. 3370–3379, Apr. 2021.

[4] R. Yahiaoui, F. Alilat, and S. Loumi, "Parallelization of fuzzy ARTMAP architecture on FPGA: Multispectral classification of ALSAT-2A images," *IEEE Trans. Ind. Electron.*, vol. 64, no. 12, pp. 9487–9495, Dec. 2017.

[5] B. Zhao et al., "Unleashing the potential of on-chip AI-powered hyperspectral hardware computing—A tutorial," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 71, no. 3, pp. 1733–1737, Mar. 2024.

[6] D.-D. Zhao, W.-B. Mao, P. Chen, Y.-J. Dang, and R.-H. Liang, "FPGA-based real-time synchronous parallel system for underwater acoustic positioning and navigation," *IEEE Trans. Ind. Electron.*, vol. 71, no. 3, pp. 3199–3207, Mar. 2024.

[7] J. Ye et al., "Edge computing accelerator for real-time defect detection of photovoltaic panel on lightweight FPGAs," *IEEE Trans. Instrum. Meas.*, vol. 74, pp. 1–15, 2025.

[8] C. Gonzalez, J. Resano, A. Plaza, and D. Mozos, "FPGA implementation of abundance estimation for spectral unmixing of hyperspectral data using the image space reconstruction algorithm," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 5, no. 1, pp. 248–261, Feb. 2012.

[9] R. Macias, S. Bernabé, D. Báscones, and C. González, "FPGA implementation of a hardware optimized automatic target detection and classification algorithm for hyperspectral image analysis," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[10] P. Palacios, D. Báscones, C. González, and D. Mozos, "A real-time FPGA implementation of the LCMV algorithm for target classification in hyperspectral images using LDL decomposition," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, pp. 1–14, 2024.

[11] B. Morcillo, D. Báscones, C. González, J. M. Mendías, and D. Mozos, "Parametric pipelined k-means implementation for hyperspectral processing on spacecraft embedded FPGA," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 17, pp. 15927–15941, 2024.

[12] W. Hu, Y. Huang, L. Wei, F. Zhang, and H. Li, "Deep convolutional neural networks for hyperspectral image classification," *J. Sensors*, vol. 2015, no. 1, 2015, Art. no. 258619.

[13] R. Hang, Q. Liu, D. Hong, and P. Ghamisi, "Cascaded recurrent neural networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 8, pp. 5384–5394, Aug. 2019.

[14] D. Hong et al., "SpectralFormer: Rethinking hyperspectral image classification with transformers," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–15, 2022.


Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:17:17 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.




[15] L. Sun, G. Zhao, Y. Zheng, and Z. Wu, "Spectral–spatial feature tokenization transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–14, 2022.

[16] Z. Zhao, X. Xu, S. Li, and A. Plaza, "Hyperspectral image classification using groupwise separable convolutional vision transformer network," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, pp. 1–17, 2024.

[17] R. Pitonak, J. Mucha, L. Dobis, M. Javorka, and M. Marusin, "CloudsatNet-1: FPGA-based hardware-accelerated quantized cnn for satellite on-board cloud coverage classification," *Remote Sens.*, vol. 14, no. 13, 2022, Art. no. 3180.

[18] W. He, Y. Yang, S. Mei, J. Hu, W. Xu, and S. Hao, "Configurable 2D–3D CNNs accelerator for FPGA-based hyperspectral imagery classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 16, pp. 9406–9421, 2023.

[19] A. Baba and T. Bonny, "FPGA-based parallel implementation to classify hyperspectral images by using a convolutional neural network," *Integration*, vol. 92, pp. 15–23, Sep. 2023.

[20] I. Pitas, *Digital Image Processing Algorithms and Applications.* Hoboken, NJ, USA: Wiley, 2000.

[21] B. J. Kang et al., "A survey of FPGA and ASIC designs for transformer inference acceleration and optimization," *J. Syst. Archit.*, vol. 155, 2024, Art. no. 103247.

[22] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, and K. Keutzer, "I-BERT: Integer-only BERT quantization," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2021, pp. 5506–5518.

[23] N. P. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," in *Proc. ACM/IEEE 44th Annu. Int. Symp. Computer Archit. (ISCA),* 2017, pp. 1–12.

[24] H. Genc et al., "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *Proc. 58th ACM/IEEE Des. Autom. Conf. (DAC)*, 2021, pp. 769–774.

[25] A. Samajdar, J. M. Joseph, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, "A systematic methodology for characterizing scalability of DNN accelerators using scale-sim," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw. (ISPASS)*, 2020, pp. 58–68.

[26] F. Melgani and L. Bruzzone, "Classification of hyperspectral remote sensing images with support vector machines," *IEEE Trans. Geosci. Remote Sens.*, vol. 42, no. 8, pp. 1778–1790, Aug. 2004.

[27] M. Dalponte, H. O. Ørka, T. Gobakken, D. Gianelle, and E. Næsset, "Tree species classification in boreal forests with hyperspectral data," *IEEE Trans. Geosci. Remote Sens.*, vol. 51, no. 5, pp. 2632–2645, May 2013.

[28] L. Ma, M. M. Crawford, and J. Tian, "Local manifold learning-based $k$-nearest-neighbor for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 48, no. 11, pp. 4099–4109, Nov. 2010.

[29] W. Zhao and S. Du, "Spectral–spatial feature extraction for hyperspectral image classification: A dimension reduction and deep learning approach," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 8, pp. 4544–4554, Aug. 2016.

[30] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Oct. 2016.

[31] T. Wang et al., "ViA: A novel vision-transformer accelerator based on FPGA," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 41, no. 11, pp. 4088–4099, Nov. 2022.

[32] P. Dong et al., "HeatViT: Hardware-efficient adaptive token pruning for vision transformers," in *Proc. IEEE Int. Symp. High-Perform. Computer Archit. (HPCA)*, 2023, pp. 442–455.

[33] D. Parikh et al., "Accelerating vit inference on FPGA through static and dynamic pruning," in *Proc. IEEE 32nd Annu. Int. Symp. Field-Programmable Custom Comput. Mach. (FCCM)*, 2024, pp. 78–89.

**Hui Zhang** (Member, IEEE) received the B.S., M.S., and Ph.D. degrees in pattern recognition and intelligent systems from Hunan University, Changsha, China, in 2004, 2007, and 2012, respectively.

From 2017 to 2018, he was a Senior Visitor with the Department of Electrical and Computer Engineering, University of Windsor, Windsor, ON, Canada. He is currently a Professor with the School of Artificial Intelligence and Robotics, Hunan University, where he is also the Deputy Director of the National Engineering Research Center of Robot Visual Perception and Control Technology. His research interests include machine vision, image processing, and robot vision.

**Hai Wang** received the B.S. degree in automation from Wuhan University of Technology, Wuhan, China, in 2019. He is currently working toward the M.S. degree in control science and engineering from the School of Artificial Intelligence and Robotics, Hunan University, Changsha, China.

His research interests include hyperspectral image processing, deep learning, and hardware acceleration.

**Bingrui Zhao** (Student Member, IEEE) received the B.S. degree in electronic and information engineering from the Northeastern University, Shenyang, China, in 2017, and the M.S. degree in circuit and system from Xiamen University, Xiamen, China, in 2020. He is currently working toward the Ph.D. degree in control science and engineering with the School of Artificial Intelligence and Robotics, Hunan University, Changsha, China.

His research interests include industrial defect detection, neural network compression, and hardware acceleration.

**Tongzhi Niu** received the B.E. degree in mechanical engineering from the School of Mechanical and Electrical Engineering, Wuhan University of Technology, Wuhan, China, in 2018, and the Ph.D. degree in mechanical science and engineering from the School of Mechanical Science and Engineering, Huazhong University of Science and Technology, Wuhan, in 2025.

From 2023 to 2024, he was a Visiting Student with the Department of Engineering, National University of Singapore, Singapore. He serves as an Assistant Researcher with the School of Artificial Intelligence and Robotics, Hunan University. His research interests include image processing and defect detection.

**Yunkang Cao** (Graduate Student Member, IEEE) received the B.S. and Ph.D. degrees in mechanical engineering from Huazhong University of Science and Technology (HUST), Wuhan, China, in 2020 and 2025, respectively.

He is currently an Assistant Professor with the School of Artificial Intelligence and Robotics, Hunan University, Changsha, China. His research interests include visual anomaly detection and industrial embodied intelligence, with a particular focus on building robust and intelligent systems for industrial applications. He has published over 20 papers in leading journals and international conferences, with more than 700 citations and an H-index of 12.

**Yaonan Wang** (Senior Member, IEEE) received the Ph.D. degree in electrical engineering from Hunan University, Changsha, China, in 1994.

He was a Postdoctoral Research Fellow with the Normal University of Defense Technology, Changsha, from 1994 to 1995. From 2001 to 2004, he was a Visiting Professor with the University of Bremen, Bremen, Germany. From 2001 to 2020, he was the Dean of the College of Electrical and Information Engineering, Hunan University. Since 1995, he has been a Professor with Hunan University, where he is currently the Principle Leader of the National Engineering Research Center of Robot Visual Perception and Control Technology. His research interests include image processing, intelligent control, and robotics.

Dr. Wang was a Senior Humboldt Fellow in Germany from 1998 to 2000. He is a fellow of Chinese Academy of Engineering. He is the President of China Society of Image and Graphics, Beijing, China.

Authorized licensed use limited to: Indian Inst of Inform Technology Guwahati. Downloaded on February 16,2026 at 06:17:17 UTC from IEEE Xplore. Restrictions apply.